

# Orquestração de Pipelines

Apache Airflow · dbt · Snowflake

Data Integration e Pipelines — Aula 4

MBA+ Data Engineering

Prof. Rafael S Novo Pereira



# A jornada até aqui: 4 aulas, 4 ferramentas



Aula 1

**Databricks**

Pipeline  
Bronze/Silver/Gold  
com PySpark



Aula 2

**Snowflake**

SQL, Tasks, Streams  
Azure Blob externo



Aula 3

**dbt Cloud**

Modelos versionados  
Testes de qualidade



Aula 4

**Airflow**

Orquestração de  
ponta a ponta

*Cada ferramenta é excelente no que faz. Mas nenhuma, sozinha, resolve o fluxo completo.*

# Sem orquestrador vs com orquestrador

## Sem orquestrador

- ✗ Aluno aperta play manualmente
- ✗ Se falha às 3h, ninguém sabe
- ✗ Sem retry automático
- ✗ Sem log centralizado
- ✗ Cada ferramenta isolada
- ✗ "Funciona no meu notebook"

## Com Apache Airflow

- ✓ Pipeline roda no schedule definido
- ✓ Alerta por email/Slack se falhar
- ✓ Retry automático configurável
- ✓ Logs de cada task no browser
- ✓ DAG define ordem e dependências
- ✓ Orquestra Snowflake, dbt, APIs

# Recap — Snowflake (Aula 2)

O armazém de dados na nuvem

- ▶ **Storage e compute separados.** O armazenamento é barato e independente do poder de processamento.
- ▶ **Warehouse elástico.** Liga, processa, desliga — você paga só pelo que roda.
- ▶ **SQL puro, escala absurda.** Bilhões de linhas sem gerenciar servidor.
- ▶ **Mas...** quem transforma o dado bruto em algo confiável? E quem coordena isso?

## Analogia

A biblioteca central: o acervo (storage) fica num lugar; as salas de leitura (compute) abrem e fecham conforme a demanda.

## O que ficou pronto

Dados carregados e consultáveis no warehouse. Base sólida — mas ainda cru e manual.

# Recap — dbt (Aula 3)

A camada que transforma e constrói confiança

- ▶ **Transformação como código.** SQL modular, versionado em Git, com dependências resolvidas sozinhas.
- ▶ **Testes embutidos.** unique, not\_null, accepted\_values, relationships — se falha, o pipeline para.
- ▶ **Lineage automático.** O grafo que mostra de onde cada número veio.
- ▶ **Camadas.** staging → intermediate → marts, do bruto ao pronto-para-negócio.

## A diferença que importa

'Transformei o dado' é fácil. 'Confio no dado' exige teste, lineage e disciplina — é isso que o dbt entrega.

## O que faltou na prática

Demos overview e criamos o projeto. Hoje fazemos o pipeline dbt do zero, completo — e depois plugamos no Airflow.

# O que falta: um maestro

- ▶ O Snowflake guarda. O dbt transforma. Mas quem dá o start?
- ▶ Quem garante a ORDEM: primeiro baixar os dados, depois carregar, depois transformar, depois validar?
- ▶ Quem roda isso todo dia às 6h sem ninguém clicar?
- ▶ Quem avisa quando algo quebra, e tenta de novo sozinho?

A resposta:

## Orquestração

- ▶ Agenda (quando roda)
- ▶ Sequencia (em que ordem)
- ▶ Monitora (deu certo?)
- ▶ Recupera (tenta de novo, alerta)



dbt do zero

1

FIAP MBA+

# dbt em uma frase

**dbt** = o **T** do ELT. Você escreve **SELECTs**, o dbt vira tabelas e views no Snowflake — testadas e documentadas.

## Onde roda

O dbt NÃO processa dados. Ele gera SQL e manda o Snowflake executar. Todo o peso é do warehouse.

## source() vs ref()

source() = tabela que veio de fora. ref() = saída de outro model seu. É o ref() que monta o grafo.

## O comando

dbt build = roda models + testes na ordem certa. Se um teste falha, para o que vem depois.



# As 4 camadas = arquitetura Medallion

O mesmo padrão Bronze / Silver / Gold, em nomes dbt

|                     |                         |                                                                     |
|---------------------|-------------------------|---------------------------------------------------------------------|
| <b>sources</b>      | <b>Origem declarada</b> | As tabelas que vêm de fora (TPC-H / Olist). Não copia nada.         |
| <b>staging</b>      | <b>Bronze</b>           | Padroniza: renomeia, tipa, 1:1 com a fonte. Materializa como view.  |
| <b>intermediate</b> | <b>Silver</b>           | JOINS e limpeza: junta pedidos + itens + clientes. Vira table.      |
| <b>marts</b>        | <b>Gold</b>             | Agregados de negócio: receita por estado, ticket médio. Vira table. |



# Camada 1 — Sources

Declarar a origem, não duplicá-la

- ▶ **Sources apontam para tabelas externas.** Você diz 'isso existe lá', e o dbt passa a rastrear.
- ▶ **Zero cópia.** Nenhum dado é movido — só referenciado.
- ▶ **Vira a base do lineage.** Todo model que usa source() fica ligado à origem.

```
# _olist__sources.yml
version: 2
sources:
  - name: olist
    database: OLIST_LAB
    schema: BRONZE
    tables:
      - name: orders
      - name: order_items
      - name: customers
```

# Camada 2 — Staging (Bronze)

Padroniza o bruto: renomeia, tipa, calcula

- ▶ **1:1 com a fonte.** Uma view de staging por tabela de origem.
- ▶ **Renomeia para PT-BR.** order\_id → pedido\_id, etc.
- ▶ **View, não table.** Não ocupa storage; recalcula sob demanda.

```
-- stgolist__pedidos.sql
with src as (
  select * from
    {{ source('olist','orders') }}
)
select
  order_id      as pedido_id,
  customer_id   as cliente_id,
  order_status  as status
from src
```

# Camada 3 — Intermediate (Silver)

O JOIN que monta a visão de negócio

- ▶ **Junta as peças.** pedidos + itens + clientes num grão só.
- ▶ **Calcula derivados.** dias\_até\_entrega, valor líquido.
- ▶ **Vira table.** Pesado de remontar — materializa de verdade.
- ▶ **ref(), não source().** Aqui você consome seus próprios models.

```
-- int_pedidos_itens.sql
select
  p.pedido_id,
  c.estado,
  i.valor_liquido,
  datediff('day',
    p.data_compra,
    p.data_entrega) as dias
from {{ ref('stgolist__pedidos') }} p
join {{ ref('stgolist__itens') }} i ...
```

# Camada 4 — Marts (Gold)

Pronto para dashboard e analista

- ▶ **Agregados de negócio.** Receita por estado, ticket médio, taxa de atraso.
- ▶ **É o que o BI consome.** Analista nunca toca em staging ou source.
- ▶ **Regra de ouro.** JOIN no staging? sobe pra intermediate. Lógica de negócio no intermediate? sobe pra marts.

```
-- fct_receita_estado.sql
select
  estado,
  count(distinct pedido_id)
    as total_pedidos,
  sum(valor_liquido)
    as receita,
  avg(dias) as media_entrega
from {{ ref('int_pedidos_itens') }}
group by estado
```

# Testes de qualidade

O que separa 'transformei' de 'confio'

## unique

não há pedido\_id duplicado

## not\_null

campo obrigatório nunca vazio

## accepted\_values

status só pode ser valores válidos

## relationships

todo item pertence a um pedido real

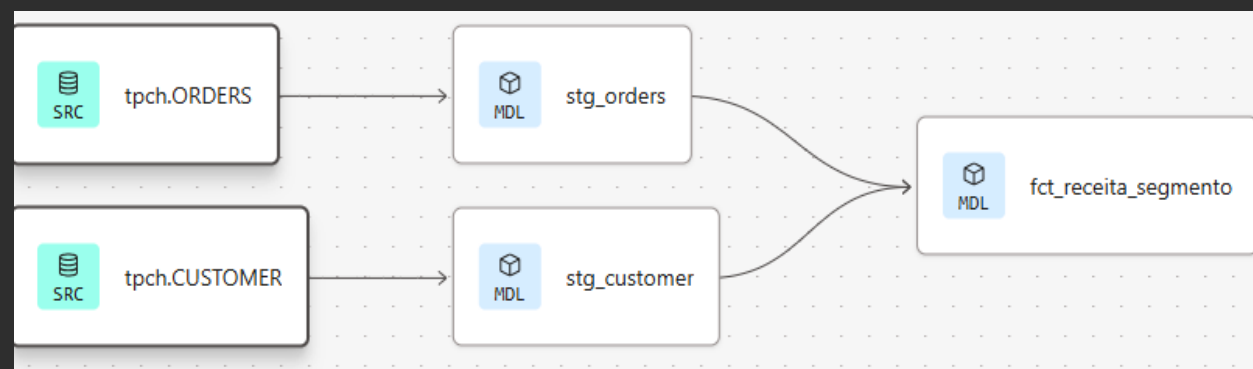
⚠ No dbt Fusion, `accepted_values` e `relationships` exigem os parâmetros aninhados sob `arguments`:

```
- name: status
  data_tests:
    - accepted_values:
        arguments:          # <- obrigatório no Fusion
          values: ['entregue', 'enviado', 'cancelado']
```

# Lineage — rastreabilidade

De onde saiu esse número no dashboard?

- ▶ **O grafo de dependências.** source → staging → intermediate → marts, desenhado sozinho.
- ▶ **Gerado pelo ref().** Você nunca diz a ordem; o dbt deduz.
- ▶ **No Fusion: aba Lineage do IDE.** Ao vivo, sem 'dbt docs generate' (que não existe mais).
- ▶ **É o momento mais forte.** Mostra visualmente todo o pipeline construído.



*cada seta é uma dependência que o dbt resolveu sozinho*

# dbt build vs dbt run

O comando certo para produção

## dbt run

Só os models (views/tables). Rápido para iterar em dev. NÃO roda testes.

Risco: rodar um mart isolado sem materializar o intermediate → 'object does not exist'.

## dbt build ✓

Models + testes + seeds + snapshots, na ordem do DAG. Se um teste falha, para o downstream.

É o comando de produção e CI. É o que o Airflow vai chamar.

**Regra: em dev, run para velocidade; em pipeline, sempre build.**



# Airflow



# 2

FIAP MBA+

# Quem usa Airflow no Brasil?

E no mundo: Spotify, Uber, Netflix, Airbnb, Twitter...



**Nubank**

1000+ DAGs processando  
dados de 90M+ clientes



**Itaú**

Pipelines de risco e  
compliance regulatório



**Mercado Livre**

Orquestra dados de  
milhões de transações/dia

*Airflow é o padrão de mercado. Saber Airflow hoje é como saber SQL há 10 anos.*

# O que é orquestração?

A torre de controle de tráfego aéreo

- ▶ **Vários voos, uma torre.** A torre não pilota — ela coordena quem decola, quando e em que ordem.
- ▶ **Dependências.** Avião B só decola depois que A pousa. O Airflow faz isso com tasks.
- ▶ **Se algo falha.** A torre redireciona, tenta de novo, alerta. Retries e alertas no Airflow.
- ▶ **Horário.** Tudo roda no schedule — não no 'quando alguém lembrar'.

## A regra de ouro

Airflow é o ORQUESTADOR, não o motor de execução.

Não coloque lógica pesada nele. Deixe o dbt transformar e o Snowflake computar. O Airflow só coordena.

# DAG, task, operator

O vocabulário do Airflow

## DAG

Directed Acyclic Graph. O desenho do pipeline: tasks + dependências, sem ciclos. Um arquivo Python.

## Task

Uma unidade de trabalho. Um nó do grafo. Ex.: 'carregar Bronze', 'rodar dbt'.

## Operator

O molde de uma task. SQLExecuteQueryOperator, PythonOperator, BashOperator.

No Airflow 3.x você importa de `airflow.sdk` — `@dag`, `@task`



# Idempotência

O conceito que torna pipelines confiáveis

Rodar a mesma task 1 ou 10 vezes produz **o mesmo resultado**.

## Analogia: o botão do elevador

Apertar 5 vezes não chama 5 elevadores. O estado final é o mesmo. Idempotência é isso.

## Por que importa

Se um pipeline falha na metade e você re-roda, não pode duplicar dados. dbt e bons DAGs são idempotentes por design.

# O que mudou no Airflow 3.x

Não é upgrade — é reescrita

- ▶ **Task SDK.** Autoria via `airflow.sdk`; tasks rodam isoladas, com mais segurança.
- ▶ **DAG Versioning.** Cada execução fica amarrada à versão do código que a iniciou.
- ▶ **Datasets → Assets.** Agendamento orientado a evento (event-driven).
- ▶ **Nova UI.** React + FastAPI, visões por task e por asset.

## Versão

Airflow 3.x (a série estável atual chegou ao 3.2). Requer Python 3.10+. Use 'Airflow 3.x' nos materiais para não datar.

## Atenção

`SnowflakeOperator` foi REMOVIDO. Use `SQLExecuteQueryOperator`.

# Arquitetura do Airflow

Analogia: uma cozinha profissional



## Webserver

*O menu*

Interface gráfica (UI)



## Scheduler

*O chef*

Decide o que rodar e quando



## Executor

*Os cozinheiros*

Executa as tasks (pods K8s)



## Metadata DB

*O caderno*

Histórico, estado, configs

# Conexão e segurança

Onde o Airflow guarda as credenciais do Snowflake

- ▶ **Connection no Airflow.** Tipo Snowflake: login, senha, schema + Extra JSON.
- ▶ **Um lugar só.** O Cosmos lê dessa connection — você não repete credencial.
- ▶ **Produção = key pair.** Desde nov/2025 o Snowflake bloqueia senha de fator único em acesso programático.

```
# Connection: snowflake_default
login:      RAFAELNOVO
password:    ****
schema:     PUBLIC

# Extra (JSON)
{
  "account": "AJQWLVN-XJC53275",
  "warehouse": "COMPUTE_WH",
  "database": "OLIST_LAB",
  "role":      "ACCOUNTADMIN"
}
```





# Conectar tudo

# 3

FIAP MBA+

# ELT moderno

Por que esta stack é o padrão da indústria



Com warehouse barato e elástico, carrega-se o dado cru primeiro e transforma-se DENTRO do warehouse.

## Snowflake

Storage + compute. Faz o trabalho pesado do SQL.

## dbt

O 'T'. Transforma, testa, documenta. Versionado em Git.

## Airflow

Orquestra: ingestão → dbt → BI. Agenda e monitora.

# astronomer-cosmos

A cola entre dbt e Airflow

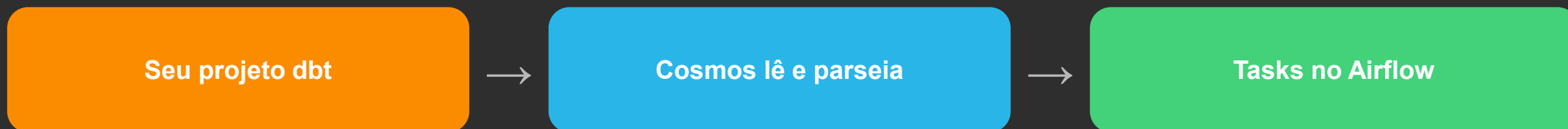
- ▶ **Cada model dbt vira uma task.** O Cosmos lê seu projeto e gera o grafo no Airflow.
- ▶ **Observabilidade por model.** Falhou? você vê qual model e re-roda só ele.
- ▶ **Adeus caixa-preta.** Antes era um BashOperator opaco; agora cada run+test aparece.
- ▶ **Padrão de mercado.** 21M+ downloads/mês; é o jeito open-source de rodar dbt no Airflow.

```
from cosmos import (DbtDag,
                    ProjectConfig, ProfileConfig)

dag = DbtDag(
    dag_id="dbt_olist",
    project_config=ProjectConfig(
        "/usr/local/airflow/dbt/olist"),
    profile_config=profile,
    schedule="@daily",
)
```

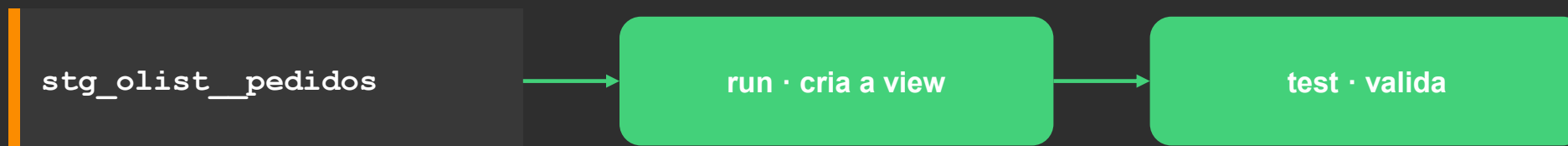
# Cosmos — como funciona por dentro

De projeto dbt a grafo do Airflow, automaticamente



O Cosmos descobre seus models e o que cada `ref()` depende — e desenha o grafo sozinho. Você não escreve nenhuma dependência à mão.

Cada model vira um par de tasks:



# Cosmos — anatomia do código

As quatro peças do DbtTaskGroup

```
from cosmos import (DbtTaskGroup,
                    ProjectConfig, ProfileConfig,
                    ExecutionConfig)

transform = DbtTaskGroup(
    group_id='dbt_transform',
    project_config=ProjectConfig(
        '/.../dags/dbt/olist'),
    profile_config=profile_config,
    execution_config=ExecutionConfig(
        dbt_executable_path=DBT_EXEC),
)
```

## ProjectConfig

Onde está o projeto dbt.

## ProfileConfig

Como conectar no Snowflake.

## ExecutionConfig

Qual binário dbt usar (venv).

## group\_id

O nome do grupo no grafo.

# Cosmos — a Connection vira profile

Credencial num lugar só, gerada na hora

- ▶ **Você não escreve profiles.yml.** O ProfileMapping lê a Connection do Airflow.
- ▶ **Uma fonte de verdade.** A credencial mora só na Connection snowflake\_default.
- ▶ **Gerado on-the-fly.** O profile do dbt é montado na hora de rodar.
- ▶ **Seguro.** Senha não fica em texto no repositório.

Connection snowflake\_default



ProfileMapping (Cosmos)



profiles.yml gerado → dbt conecta

# Cosmos vs BashOperator

Por que vale a pena — e o Plano B

## BashOperator (Plano B)

Uma caixa só: dbt build roda tudo junto.

Falhou? você não vê qual model.

Precisa de um profiles.yml próprio.

Simples — boa rede de segurança.

## Cosmos (recomendado)

Cada model é uma task no grafo.

Falhou? vê o model exato e re-roda só ele.

Lê a Connection do Airflow.

Lineage + orquestração na mesma tela.

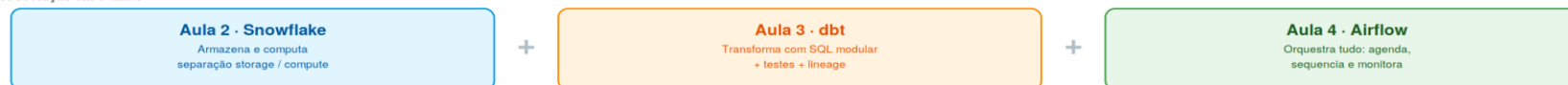
# A arquitetura final

Tudo conectado: um pipeline, ponta a ponta

## FIAP A Jornada Completa — Snowflake → dbt → Airflow

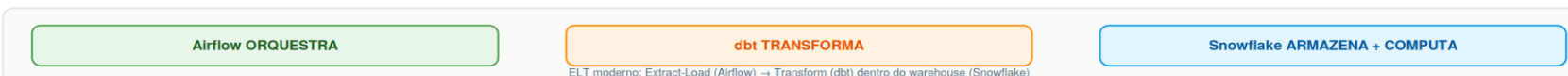
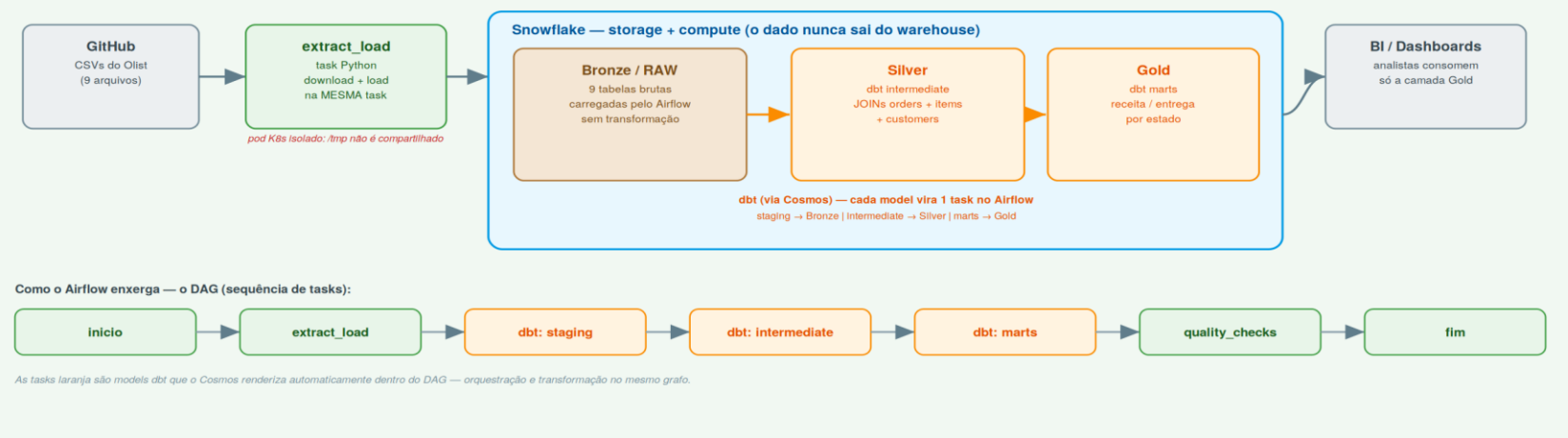
Como a arquitetura de dados cresce, aula a aula, até um pipeline orquestrado ponta a ponta.

A evolução em 3 aulas



### Apache Airflow 3.x — orquestração - a arquitetura final do pipeline Olist

O Airflow agenda e monitora; cada etapa abaixo é uma task no DAG.



Legenda: ■ Airflow ■ dbt ■ Snowflake ■ Fonte / Consumo

Prof. Rafael S Novo Pereira · FIAP MBA+ — Data Integration e Pipelines · Aula 4

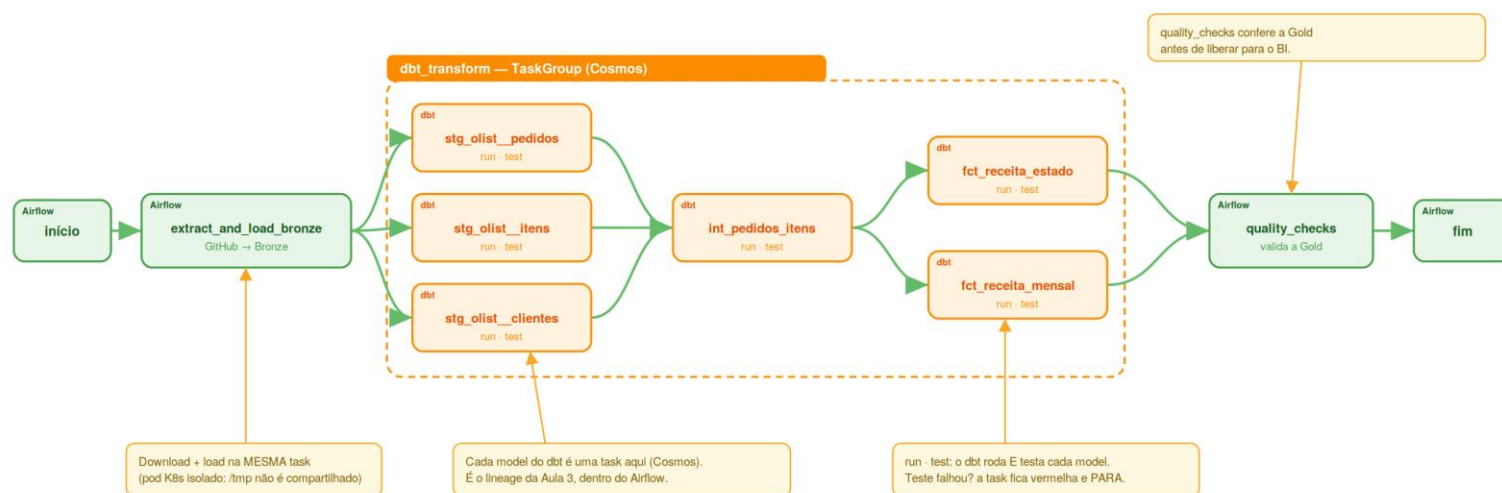


# O grafo que o Cosmos gera

É isto que você vê no Airflow → aba Graph

## FIAP A visão Graph do DAG — pipeline\_olist\_completo

Onde você vê a jornada inteira executando. No Airflow (Astro): abra o DAG → aba Graph. Cada caixa abaixo é uma task que fica verde sozinha.



O que acontece com o dado, em paralelo:



Legenda: ■ task do Airflow ■ model do dbt (Cosmos) ■ sucesso

Prof. Rafael S Novo Pereira · FIAP MBA+ — Data Integration e Pipelines · Aula 4

# A jornada executando — passo a passo

O que a turma vê acontecer, em ordem

- |   |                                      |                                                            |
|---|--------------------------------------|------------------------------------------------------------|
| 1 | <code>Trigger no Airflow</code>      | o DAG começa; daqui ninguém clica em nada                  |
| 2 | <code>extract_and_load_bronze</code> | baixa 9 CSVs do GitHub → Snowflake Bronze (mesma task)     |
| 3 | <code>staging (dbt)</code>           | 3 models viram tasks e padronizam o bruto, em paralelo     |
| 4 | <code>int_pedidos_itens (dbt)</code> | o JOIN; só roda depois do staging — o dbt resolveu sozinho |
| 5 | <code>marts (dbt)</code>             | receita por estado e por mês — a camada Gold               |
| 6 | <code>quality_checks</code>          | confere a Gold antes de liberar para o BI                  |
| 7 | <code>fim</code>                     | tudo verde; valide no Snowflake — SP lidera a receita      |

# O pipeline Olist

O que o DAG faz, etapa por etapa

**inicio**

marca o começo

**extract\_load**

baixa 9 CSVs do GitHub e carrega no Snowflake (Bronze)

**dbt (Cosmos)**

staging → Silver → Gold, cada model uma task

**quality\_checks**

valida os números antes de liberar

**fim**

pipeline concluído

# A pegadinha do Kubernetes

Por que download e load viram UMA task

- ▶ **Cada task = um pod isolado.** No KubernetesExecutor, toda task roda no próprio container efêmero.
- ▶ **O /tmp não é compartilhado.** O arquivo que a task A baixou some quando a task B sobe.
- ▶ **Logo: extract + load atômico.** Baixar o CSV e carregar no Snowflake têm que estar na MESMA task.
- ▶ **Lição de vida real.** Esse erro é clássico — e justifica desenhar a ingestão como uma etapa só.

## O que NÃO fazer

Task 1 baixa para /tmp/dados.csv.

Task 2 tenta ler /tmp/dados.csv → não existe.

O pod da task 1 já morreu e levou o /tmp junto.

# O resultado

Dados reais, orquestrados sem ninguém tocar no Snowflake

R\$ 6 mi

receita em SP — o estado líder

41.746

pedidos em São Paulo

~12,9%

taxa de atraso no RJ

Esses números foram baixados do GitHub, carregados no Snowflake, transformados em Bronze/Silver/Gold pelo dbt e validados — **tudo orquestrado pelo Airflow.**

# O que vocês construíram

- ▶ **Ambiente Airflow gerenciado** na nuvem (Astro), rodando em Kubernetes
- ▶ **Repositório Git conectado** com o código do pipeline versionado
- ▶ **Conexão segura ao Snowflake** credenciais guardadas no Airflow
- ▶ **Ingestão** → **Bronze** 9 CSVs carregados como tabelas brutas
- ▶ **dbt** → **Silver/Gold** JOINS e métricas, via Cosmos, como tasks
- ▶ **Qualidade + orquestração** testes rodando, tudo agendável e monitorado

*Na vida real muda só a fonte (API/blob), o schedule (diário) e o alerta (Slack). O conceito é idêntico.*

# Data Integration e Pipelines



16 horas — o que vocês construíram:



- **2 plataformas de dados**
- **Transformação com dbt**
- **Orquestração com Airflow**
- **Dados reais brasileiros**
- **Kubernetes por baixo**

Databricks (PySpark) + Snowflake (SQL)

Modelos SQL, testes, documentação

DAGs, K8s, operação em produção

Olist (100K pedidos) + TPC-H (60M rows)

Containers isolados, escaláveis, eficientes